OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (OSSP)

LINUX DEBUGGING SYSTEM

KLH Bachupally Campus · Computer Science and Engineering · PBL 2026–27

TEAM MEMBERS	ROLL NUMBERS
U. PRANEETH	2520030188
M. HARSHITH	2520030573
U. PRATHIK	2520030034

ABSTRACT

A Linux-based diagnostic toolkit developed in C to help users inspect operating-system activity. It brings process inspection, system-resource information, scheduling simulation, inter-process communication and memory-leak demonstration into one command-line project.

FEASIBILITY STUDY

Technical: Uses standard Linux and POSIX facilities with GCC/Make. Economic: Open-source tools; no hosting cost required. Operational: Menu-driven terminal interface; some diagnostics depend on Linux permissions. Schedule: Design → implementation → testing → documentation and demonstration.

PROBLEM STATEMENT & OBJECTIVES

Problem: Inspecting processes and system resources manually requires multiple commands and tools. A beginner-friendly, unified diagnostic workflow is useful for understanding Linux internals.
Objectives: (1) Display system and resource information. (2) Demonstrate process, IPC, signal and thread concepts. (3) Provide scheduling and memory-leak demonstrations with diagnostic reports.
Scope: Educational command-line prototype for Ubuntu/Linux; not a replacement for production debuggers.

EXISTING VS PROPOSED SYSTEM

Existing approach: Separate terminal commands and manual inspection; limited consolidated reporting.
Proposed system: One C-based menu combines system overview, memory/disk details, PID inspection, process listing, Round Robin simulation, IPC, signal/thread demos and report generation.

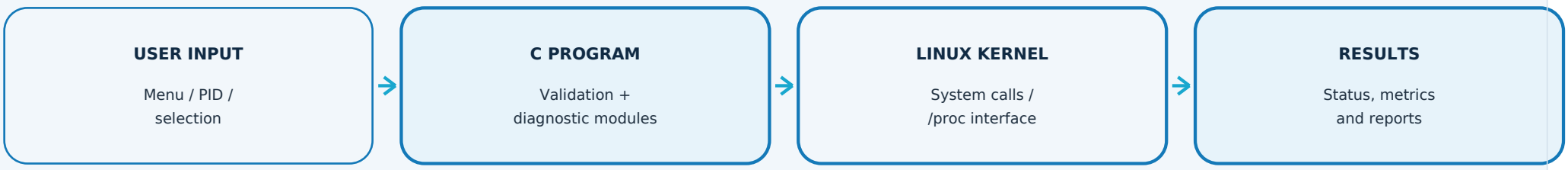
TOOLS & TECHNOLOGIES

Language: C Platform: Ubuntu / Linux (native or WSL)
Concepts: Processes, scheduling, IPC, signals, threads, memory and files
Tools: GCC, Make, Linux /proc, POSIX APIs, GDB, Git & GitHub

PROCESS, IPC & RESOURCE DESIGN

Process / resource design: The program accepts a menu choice, invokes the relevant Linux or POSIX interface, checks return values, and presents results. Process demonstrations use parent/child execution and IPC where selected; thread demonstrations illustrate shared-resource synchronization.

SYSTEM WORKFLOW / ARCHITECTURE



Input → application logic → kernel services → diagnostic output

IMPLEMENTED MODULES & RESULTS / TESTING

FEATURE	WHAT IT DEMONSTRATES	EXPECTED OBSERVATION
System overview / memory / disk	Linux system and resource information	OS, memory and storage details
PID inspection / process list	Process identification and status	Process details for selected PID
Round Robin simulation	Time quantum and CPU scheduling	Completion, turnaround and waiting times
IPC / signals / threads	Pipe, process communication and synchronization	Communication and coordinated execution
Memory leak demonstration / report	Tracked allocations and diagnostic output	Outstanding allocation summary / report file

Conclusion: The project consolidates key Linux debugging and OSSP learning activities in a single C command-line prototype. It demonstrates how user programs interact with kernel services and how processes, IPC, scheduling, memory and synchronization can be observed.

Future scope: Add richer log analysis, automated test cases, configurable monitoring intervals, and more detailed memory / process diagnostics.

References
[1] Linux man-pages project
[2] Operating Systems course material / textbook
[3] GNU GCC and GNU Make documentation

Guide: Dr. K. Anusha
GitHub: Add your team repository QR code before final print.